Built-In Transactions

When we discussed parent and child tables in *Specifying Relationships in Models*, we said that Active Record takes care of saving all the dependent child rows when you save a parent row. This takes multiple SQL statement executions (one for the parent and one each for any changed or new children).

Clearly, this change should be atomic, but until now we haven't been using transactions when saving these interrelated objects. Have we been negligent?

Fortunately, no. Active Record is smart enough to wrap all the updates and inserts related to a particular `save` (and also the deletes related to a `destroy`) in a transaction; either they all succeed or no data is written permanently to the database. You need explicit transactions only when you manage multiple SQL statements yourself.

While we've covered the basics, transactions are actually very subtle. They exhibit the so-called ACID properties: they're Atomic, they ensure Consistency, they work in Isolation, and their effects are Durable (they're made permanent when the transaction is committed). It's worth finding a good database book and reading up on transactions if you plan to take a database application live.

What We Just Did

We learned the relevant data structures and naming conventions for tables, classes, columns, attributes, IDs, and relationships. We saw how to create, read, update, and delete this data. Finally, we now understand how transactions and callbacks can be used to prevent inconsistent changes.

This, coupled with validation as described in Chapter 7, *Task B: Validation and Unit Testing*, covers all the essentials of Active Record that every Rails programmer needs to know. If you have specific needs beyond what is covered here, look to the Rails Guides[102] for more information.

The next major subsystem to cover is Action Pack, which covers both the view and controller portions of Rails.